

CH2

OOP大綱

OOP (oriented object programming)

使用物件來表示問題，並加以解決，簡化程式碼複雜度

封裝 (Encapsulation)

將實作細節隱藏、保護私有變數

繼承 (Inheritance)

可以擴充新功能、避免重複的程式碼

多型 (Polymorphism)

通過虛擬函數讓子類別覆寫函數，達到不同的行為

組合 (Composition)

在 class 中定義 class 變數，不要全部攤在第一層下面

依賴注入 (Dependency Injection)

不要在 initialize 階段建立子物件，而是將子物件放到外面建立與管理，在"注入" class 中實現組合

介面 (Interface)

只定義抽象的成員變數與方法，不直接實作

CH3 C++ 簡介

C++ 語法 - 基本 問題思考篇：

1. `printf("%s\n", 1123)`：直接把 1123 記憶體的值拿出來印
2. `\n` or `std::endl`：`\n`：大量輸出好(快) <v.s.> `std::endl` 強制緩衝區輸出 (慢)

Scope 變數範圍

使用 `::` 來分別要使用哪個函式庫的函數 e.g. `libA::func()`, `libB::func()`

使用 namespace 來指定欲使用的函式庫

```
1 #include <libA/func.h>
2 #include <libB/func.h>
3
4 namespace libA{
5 int main(){
6     func();
7     //libB::func();
8 }
9 }
```

```
1 #include <libA/func.h>
2 #include <libB/func.h>
3
4 using namespace libA;
5
6 int main(){
7     func();
8     //libB::func();
9 }
```

Compiler 的介紹

Header file (.hpp)：定義架構

file (.cpp)：定義實作方法

Include Guard (引用防護)

```
#ifndef HEADER_FILE_01
#define HEADER_FILE_01

int minus(int a, int b);

#endif // HEADER_FILE_01 02_00.h
```

```
#pragma once

int minus(int a, int b);
```

CH4 類別

指導語句 (Directives)

預處理器的指令

e.g. `#include <string> / #define LAPTOP_HPP / #endif ...`

Include Guards

避免函數或變數被重新定義

```
#ifndef LAPTOP_HPP
#define LAPTOP_HPP

...

#endif
```

This 指標

`this→name = name;` 有 `this`: 設定 `class` 的值 v.s. 沒 `this`: 被自己設定成自己(無意義)

生命週期 (Lifecycle)

程式碼區塊

```
TEST(LaptopTest, test_laptop) {
    Laptop laptop("Windows", 15.7, false, "Default Laptop", "Letni i5-8700", false);

    ASSERT_EQ("Letni i5-8700", laptop.GetCPUName());
}
```

對於程式碼來說，裡面的 `laptop` 在這個測試函數的程式區塊內，當離開這個程式區塊（例如測試結束）後，`laptop` 會被自動釋放

解構子

如果使用物件時有以下動作時，會在解構子裡寫程式來 `close` 或 `delete` 它：

- 開檔案
- 開網路連線
- 指標
- 動態配置記憶體 (`new`)

避免記憶體洩漏

RAII (Resource Acquisition Is Initialization)

值的檢查應該放在建構子中，而不是 GETTER

實用 STL

e.g. `std::string` / `std::vector` / `std::shared_ptr`

`std::shared_ptr`：以物件方法處理指標，自動釋放指標



CH5 封装

存取修飾字 (Access modifier : Public & Private)

	Class 外部	Class 內部	Class 的繼承者
public:	✓	✓	✓
privarte:	✗	✓	✗

從 class 外部變更 private 變數 : Getter/Setter + 設定時檢查值得合法性

封装方式 : class v.s. struct

	成員預設屬性	預設繼承方式	可以使用 template
struct:	public	public	✗
class:	private	private	✓

→ 需要被保護的程式才需要 private

CH6 繼承

抽象化 (Abstraction)

在父類別 (Parent) 定義但不實作，然後繼承到子類別 (Children) 實作

被別人繼承的類別

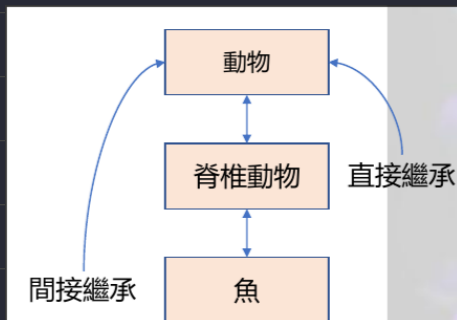
繼承別人的類

基礎類別 (Base Class) → 為衍生類別 (Derived Class)

e.g. 學生

e.g. 國小生

繼承階層圖



繼承關係

直接 (Direct) 繼承

間接 (Indirect) 繼承

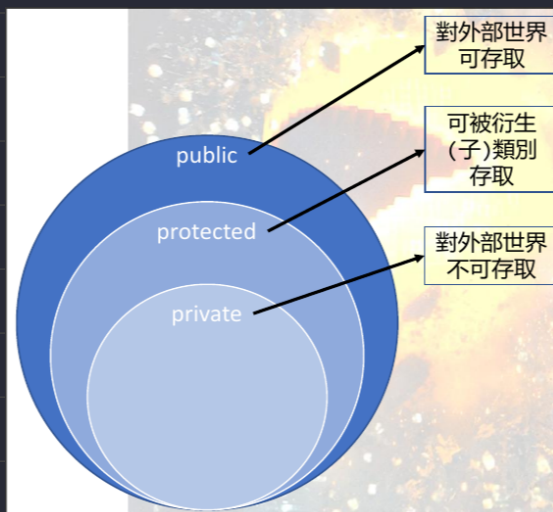
單一 (Single) 繼承：繼承自單一個基礎類別

多重 (Multiple) 繼承：繼承自多個基礎類別

初始化列表 (InitializeList)

```
1 Mage::Mage(int abilityPower, std::string name, int health) : Character(name, health){
2     this->abilityPower = abilityPower;
3 }
```

權限控制：protected



Public：所有人都可以存取

Private：是限制最嚴格的access modifier，只有class可以存取

Protected：存取是受限的，只有：

- (1) 自己
 - (2) 成員函式
 - (3) 類別的朋友 (friend)
 - (4) 子類別
- 可以存取

優點：直接存取+效率高（沒有GETTER/SETTER）

缺點：無法經由 GETTER/SETTER 驗證值的合法性

● 繼承與權限控制

		衍生類別		
		Public	Protected	Private
基礎類別	Public	Public	Protected	Private
	Protected	Protected	Protected	Private
	Private	Hidden	Hidden	Hidden

最終繼承：Final

在 class 後面加上 final 修飾字，可以避免此 class 被其他人繼承（報錯提示）

子類：覆寫函數

父類：虛擬函數

純虛擬函數 vs 虛擬函數

虛擬函數不強制一定要 override，可以採用父類的定義或子類 override 成自己的定義

純虛擬函數強制一定要 override，父類的定義不應該存在，子類一定要自己定義

CH7 多型

Overview

三種操作：Object Casting, Virtual, Virtual Destructor

多型種類：Inclusion, Ad-Hoc, Parametric

使用原因：將實作部分寫在不同的衍生類別中，避免一個函式承載過多業務邏輯

一個議題：Duck Typing

Overload vs. Override

特性	Overloading (多載)	Override (覆寫)
發生位置	同一個類別中	必須在繼承關係中
函式名稱	一樣	一樣
參數 signature	不同	完全相同
virtual 是否需要	不需要	需要
作用	讓函式能處理不同輸入	子類改寫父類行為
★ 時間點	在編譯時決定 (靜態多型)	在執行時決定 (動態多型)

Function matching vs. Function overload

編譯器會執行 Function matching 來決定 Function overload 時要執行的函式

Object Casting vs. Object Slicing

→ Object Casting：僅存取範圍受限（將 Derived Class 視為 Base Class 的動作），資料沒遺失

向上轉型（Derived → Base）：

```
1 // Create a derived shared pointer
2 // Cast that pointer to a base class pointer
3
4 std::shared_ptr<Base> b = std::make_shared<Derived>();
```

向下轉型（Base → Derived）：

```
1 std::shared_ptr<Derived> b = std::make_shared<Base>();
2
3 // This will be nullptr if casting failed
4 std::shared_ptr<Derived> d = std::dynamic_pointer_cast<Derived>(b);
```

→ Object Slicing：直接切掉 Derived Class 多的變數，直接無法再存取他們

```
1 Base b = Derived();
```

Virtual

```
1 class Example {
2     // Early Binding (編譯時綁定)
3     int function(); // 一般函數
4
5     // Late Binding (執行時綁定)
6     virtual int function(); // 虛擬函數
7     virtual int function() = 0; // 純虛擬函數
8 }
```

子類別有 **override** 該函數的父類別就一定要有 **virtual** 否則不會過編譯

如果父類別沒有 **virtual** 且子類別也沒有 **override** → 無法覆寫（因為Early binding）

Virtual Destructor（虛擬解構子）

繼承鍊中的解構子應該從衍生類別開始釋放到基本類別，如果基本類別的解構子不加 **virtual**：

1. `~Base()` 被視為 Early binding
2. 子類別的 **override** 全部無效
3. 子類別解構時直接執行基本類別函式，無法釋放子類別中的成員
4. 記憶體洩漏（Memory leak）

多型種類：Inclusion

將所有衍生類別都 `cast` 成基本類別，即可使用單一容器管理所有衍生類別的物件

背：利用父類別的型態、接收不同子類別的物件、做相同的動作、引發不同的行為

```
1 class Ore {
2     virtual std::string Smelt();
3 }
4
5 class GoldOre : public Ore {
6     std::string Smelt() override;
7 }
8
9
10 class IronOre : public Ore {
11     std::string Smelt() override;
12 }
13
14 int main() {
15     std::shared_ptr<Ore> gold = std::make_shared<GoldOre>();
16     std::shared_ptr<Ore> iron = std::make_shared<IronOre>();
17     std::vector<std::shared_ptr<Ore>> = {
18         gold, iron
19     };
20 }
```

Function Matching

當有多個同名函數時，`編譯器` 會幫你找到最符合呼叫要求的函數去執行

```
1 class Furance {
2     void addIron(IronOre ore) {...}
3     void addIron(IronOre gold) {...}
4 }
```

多型種類：Ad-Hoc

Function Overloading

這是一種 **實作技巧**，可以根據函式參數不同的 **數量** 或 **類型** 讓編譯器作 **Function matching**

```
void cropBlock(Block block, WoodAxe axe){
    /* 用木頭斧砍方塊 */
}

void cropBlock(Block block, DiamondAxe axe){
    /* 用鑽石斧砍方塊 */
}
```

```
void mine(Block block){
    /* 用手挖方塊 */
    destroyBlock(block);
}

void mine(Block block, Tool* tool){
    /* 用工具挖方塊 */
    tool->DecreaseDurability();
    destroyBlock(block);
}
```

Operator Overloading

覆寫在遇到運算元時執行的動作

```
1 class Vector2D {
2 public:
3     int x, y;
4
5     Vector2D(int x, int y) : x(x), y(y)
6     Vector2D operator+(const Vector2D& other) const {
7         return Vector2D(x + other.x, y + other.y);
8     }
9 };
10
11 int main() {
12     Vector2D v1(1, 2);
13     Vector2D v2(3, 4);
14
15     Vector2D v3 = v1 + v2;
16     return 0;
17 }
```

多型種類：Parametric

Class Template (模版)

- 靜態綁定 (Early binding)
- 只能寫在 header file 中
- 綠色方空中的 T 代表入類型會被填入到下方程式區段中的所有 T

```
1  template <typename T>
2  class Box {
3  private:
4      T content;
5
6  public:
7      // 建構子
8      Box(T val) : content(val) {}
9
10     T getContent() {
11         return content;
12     }
13
14     void setContent(T newVal) {
15         content = newVal;
16     }
17 };
18
19 int main() {
20     Box<int> intBox(100);
21 }
```

Function Template

```
1  template <typename T>
2  T findMax(T a, T b) {
3      return (a > b) ? a : b;
4  }
5
6  int main() {
7      cout << "Int:    " << findMax<int>(10, 20) << endl;
8      cout << "String: " << findMax<std::string>("A", "AB") << endl;
9  }
```

Duck Typing

使用 `template` 時編譯器會幫我們檢查傳入的類型是否以支援 `class` 的所有操作，如果不符合就會報錯

但如果有一類型符合所有 `class` 中的操作要求，但邏輯上不是預期可以丟進去 `class`，編譯器不會報錯

→ 這就是 `duck typing`

議題

`Virtual Function` 要怎麼去取得底下的實作？

要怎麼善用 `Object Casting` 來從子類變成父類型別？

為什麼我們會需要 `Virtual Destructor`？

什麼叫做 `Duck Typing`？

CH8 組合與介面

Overview

組合大概念：Class 裡面塞 Class

物件委派 (Object Delegation)：塞 Instance

聚合 (Aggregation)：叫裡面的 class 作操作

介面大概念：只宣告不實作

定義耦合 (Coupling)：繼承鍊過長

利用組合與介面取代繼承類別：一次扁平計程多個 Interface

組合與介面用於解決什麼樣的問題？

The Diamond Problem

Combining Interface

Template Issue

The suggestion of composition, interface, and inheritance.

Dependency Injection：傳入 class

TL;DR

- 聚合 (Aggregation) 與組合 (Composition) 取決於是不是委派給類別。
- 繼承鍊是一個高耦合度的實作，難以維護，透過組合與介面能夠扁平化繼承鍊，降低耦合。
- 組合與介面還能優化掉很多問題：鑽石問題、扁平化繼承、tempalte 最小需求
- 繼承可能更適合用在不需要討論耦合的情況 / 組合與介面能夠更好的應付需求與降低耦合度
- 依賴注入：根據傳入的物件是什麼來做出對應的行為

- 物件委派 (Object Delegation)

將物件的操作轉交給另一個類別實作，而非在物件中直接實作程式

```
1 class StudentGroup {
2     private:
3         std::vector<string> students; // 組合
4
5     public:
6         void addStudent(string name) {
7             students.push_back(name); // 委派
8         }
9
10        int getCount() {
11            return students.size(); // 委派
12        }
13 };
```

- 聚合 (Aggregation)

將物件在外部建立並將其指標傳入類別中，其生命週期不會因為類別銷毀而結束 (物件不會消失)

```
1 class Teacher {
2     // 老師自己有生命
3 };
4
5 class Department {
6     private:
7         Teacher* teacher; // 聚合
8
9     public:
10        Department(Teacher* t) : teacher(t) {}
11
12        ~Department() {
13            // Department 被銷毀時，我們「不」刪除 teacher。
14        }
15 };
```

- 介面大概念

介面 vs. 繼承

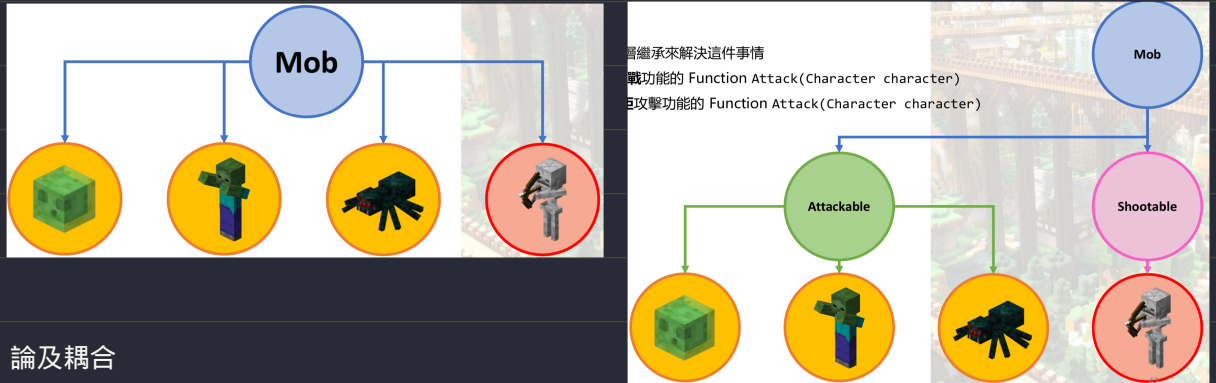
繼承：從類別出發 (父子關係)

介面：從函式出發 (有哪些功能)

- 定義耦合 (Coupling)

繼承是有「父子」衍生的概念，導致如果要新增功能就要把原本的程式大幅重構甚至使繼承鍊變長

→ 此稱為高耦合



論及耦合

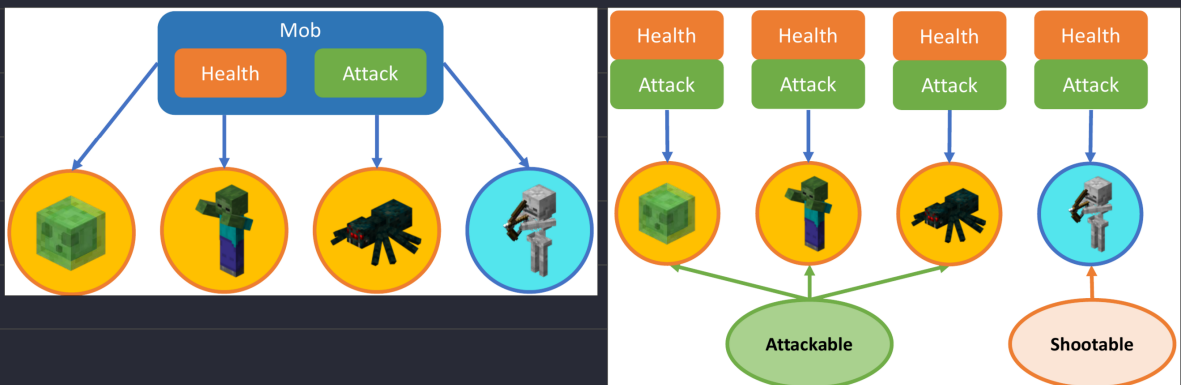
我們期望上我們的實作是低耦合度跟程式碼的重複率做衡量

如果未來不會需要新增功能：繼承

如果未來有擴展的能：介面

- 利用組合與介面取代繼承類別

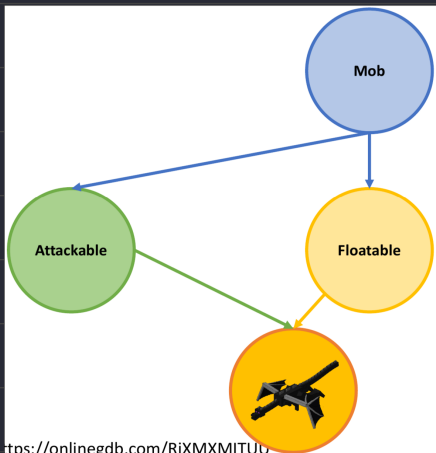
扁平化繼承鏈，如果未來要新增其他角色也不會造成需要大改程式碼



- The Diamond Problem

The Diamond Problem 是指在多重繼承中，當兩個父類別繼承自同一個基底類別，而子類別又同時繼承這兩個父類別時（形成菱形結構），會導致編譯器在存取基底類別成員時無法確定是要用哪一個分支。

透過「組合與介面」來避免 Diamond Problem 發生，以下 Code 發生 diamond problem



https://onlinegdb.com/RiXMXMITU0

```
class Mob {
public:
    virtual void action() = 0;
};

class Attackable : public Mob {
public:
    void action() override {
        std::cout << "Attack!" << std::endl;
    }
};

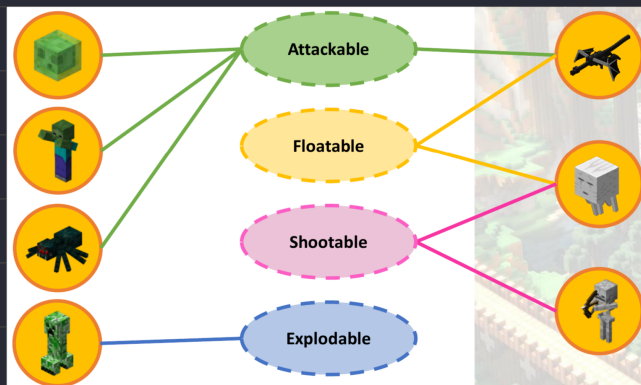
class Floatable : public Mob {
public:
    void action() override {
        std::cout << "Float!" << std::endl;
    }
};

class EnderDragon : public Attackable, public Floatable {
    // 不覆寫action()，導致歧義
};

int main() {
    EnderDragon enderdragon;
    enderdragon.action(); // 這行將會引發編譯錯誤
    return 0;
}
```

- Combining Interface

扁平化計程多個介面



- Template Issue

使用 template 的條件就是 T 要符合最小要求，否則編譯會報錯，但這樣每次都要手動檢查T有什麼要求

```
1 template <typename T>
2 class Stack {
3     std::vector<T> stack;
4 }
```

將 template 改成介面，這樣就以確保傳入的類型都有繼承 ISmelttable，且有實作就好

```
1 class Stack {
2     std::vector<ISmelttable> stack;
3 }
```


- The suggestion (該不該繼承)

- 在少數情況下，繼承會有更好的可維護性
- 我們其實可以考慮實作會不會有很頻繁的變更，來決定要不要使用繼承

繼承注意事項

- 避免讓子類能夠直接存取父類的值 (封裝性)
- 使用 Private 來權限規範好不讓子類使用的函式
- 使用 Protected 來讓子類能夠使用父類的 API

- 依賴注入 (Dependency Injection)

在建構子中將類別當作參數

好處：減少建構子參數量以增加維護性 + 耦合度變低

```
class ISmeltable {
public:
    virtual std::string Smelt() = 0;
};

class Furnace {
private:
    std::shared_ptr<ISmeltable> item;
public:
    Furnace(std::shared_ptr<ISmeltable> item){
        this->item = item;
    }
    std::string Smelt(){
        return item->Smelt();
    }
};
```

只要有 ISmeltable 都可以支援